UCLouvain

École polytechnique de Louvain

LELEC2885 Report

Mikhaïl Watchueng Tchekam **Maxime Devillet** **Edouard Barette**

Louvain-la-Neuve, Belgium

Introduction

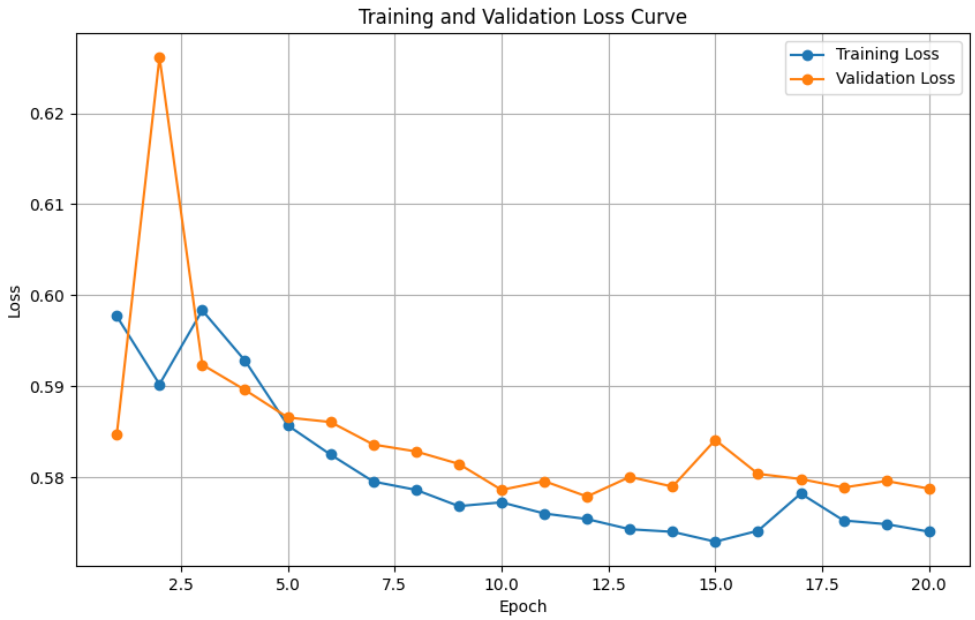
During this project, our task was to improve a basic image segmentation model designed to create a binary mask outlining the silhouettes of cats and dogs of all breeds within an image. To achieve this, we modified the model's base architecture by experimenting with our own techniques and parameters. In this report, we will describe the various techniques that allowed us to optimize our model to the fullest and analyze the results obtained.

1 Part A

1.1 Network Training

To determine the model's hyperparameters, we used a grid search between the learning rate and the batch size. We concluded that the best parameters are:

- **Learning rate** = 0.001: defined through grid search
- **Batch size** = 16: defined through grid search
- **Epochs** = 20: point where the model converges
- **Loss** = nn.BCEWithLogitsLoss(): combines a sigmoid activation with the binary cross-entropy loss for better numerical stability in binary classification tasks



Here is our graph. When we started improving our model, we had a validation loss close to 1 with particularly pronounced overfitting (the training loss was much better than the validation loss). Over time, with improvements and research, we obtained this graph showing the training loss and validation loss of our best model on images with a resolution of 64x64. Unfortunately, we were unable to display the results for 256x256 images because we no longer had access to a GPU, but the validation loss was 0.77 with a similar curve.

Here, the graph shows that the training loss and validation loss are very close to each other, which is a very good sign that the model is learning without overfitting. Then, we can observe that the loss gradually decreases until it stabilizes at epoch 16, where both curves remain steady, once again showing that the model is not overfitting on the test data. In our opinion, this graph indicates that the model is well chosen, not too large to avoid overfitting, nor too small to still produce results with low loss.

1.2 Network Architecture.

To improve performance, we replaced the basic convolutional model with a **U-Net**. The U-Net is a convolutional model consisting of an **encoder** and a **decoder**, with the unique feature of incorporating direct connections between corresponding layers of the encoder and decoder, called **skip connections**. The idea behind this model is to retain information from earlier convolutions, even if they are at different scales. This is particularly useful for our **segmentation** task, as it allows combining fine details with global information.

We proceeded as follows:

- **For the encoder:**
  - Gradually reduce the image size through convolutions while increasing the number of channels to extract increasingly complex features.
- **For the decoder:**
  1. **Resize** the current output to match the size of the previous encoder layer.
  2. **Concatenate** this resized output with the corresponding encoder output to create a **skip connection**.
  3. **Pass** the result through a convolutional block to refine the features and progressively reconstruct the image.

1.3 Qualitative Evaluation

Now that the model has been properly defined, we can observe and comment on the results produced with test images. We will then analyze the best-performing images as well as those that are less conclusive.

1.3.1 Best Results

Here, we can observe that in both images, there is very little difference between the true mask and the predicted one. Even in Figure 2, where the contrast between the background and the dog is low, which should normally challenge our model, it appears that the model still does a good job of distinguishing the shape despite the low color contrast. This suggests that the model is still able to identify the shapes well, even with minimal color differences.

1.3.2 Least Successful Results

Here are examples of less successful masks. In the second image, we can see that although the general shape of the dog is recognized, it overflows on the left side, which corresponds to a patch with a color similar to that of the dog. This leads us to hypothesize that the model learns more to draw the shape of the animal based on its color rather than its contours, thus placing too much importance on the colors.

1.4 Quantitative Evaluation

for knowing if the model learning well we will considering three key metrics: Train Loss, Validation Loss, and Validation Dice Score.

1. **Train Loss :** We use it to determine if the model is learning well on the training data. However, the validation loss is more useful for assessing whether the model has correctly understood how to perform its task on unseen data. We use it to understand how the model has evolved and to check if it is overfitting or not
2. **Validation Loss** : As explained above, the validation loss represents the true learning of the model. It is a good indicator of whether the model is performing well or not. It is the main value we looked at to determine if our modifications improved the predictions or not
3. **Validation Dice Score** :The Validation Dice Score is another important metric used to evaluate the performance of the model. It measures the overlap between the predicted segmentation and the true segmentation. A higher Dice Score indicates better performance. We use it to complement the validation loss because it specifically focuses on the accuracy of the segmentation, providing a clearer understanding of how well the model identifies and delineates objects, especially in image segmentation tasks.
4. **Image inspection:** We simply examine some images to see if the predicted mask fits well. If not, we analyze what is wrong with the prediction.

To identify if the model is learning effectively, we look at all indicators to understand the progression of our model and how to improve it.

2 Part B

2.1 Uncertainty Estimation

The main steps for this method are as follows: For each test image, generate multiple augmented versions using different transformations. Pass each augmented image through the network to obtain predicted segmentation masks. Align all predicted masks by reversing the transformations applied during augmentation. Compute the entropy of each pixel across the aligned masks to evaluate the uncertainty.

The implementation of these guidelines is encapsulated in the test_uncertainty and plot_uncertainty methods of the Network_Class in model.py.
The test_uncertainty method starts by loading the model weights for different epochs to evaluate the model's performance and uncertainty over time. For each test image, multiple augmented versions are generated using various transformations such as horizontal flip, vertical flip, rotation, blur, brightness adjustment, noise addition, contrast adjustment, saturation adjustment, and grayscale conversion. Each augmented image is passed through the network to obtain the predicted segmentation mask. The predicted masks are then realigned by reversing the transformations applied during augmentation. The aligned predicted masks are stacked together, and the softmax function is applied to convert the logits to probabilities. The entropy of each pixel across the aligned masks is computed to evaluate the uncertainty. The method calls plot_uncertainty to visualize the input image, ground truth mask, predicted masks, and the corresponding entropy maps.

The plot_uncertainty method visualizes the input image, ground truth mask, predicted masks at different epochs, and the corresponding entropy maps. This visualization helps in understanding the model's performance and the areas of high uncertainty in the predictions.
Upon evaluating the model's predictions at epochs 5 and 10 using the test time data augmentation (TTA) method, we observed a noticeable difference in the entropy maps. At epoch 5, the entropy of the predictions was relatively higher, indicating greater uncertainty in the model's segmentation masks. This higher entropy suggests that the model was less confident in its predictions, likely due to insufficient training.
By epoch 10, the entropy of the predictions had significantly decreased. The entropy maps showed more defined and lower entropy values, indicating that the model had become more confident in its predictions. This improvement in confidence can be attributed to the additional training epochs, which allowed the model to better learn the features and patterns in the data, resulting in more accurate and reliable segmentation masks.

2.2 Pre-training model

To facilitate the training of the model, an excellent solution is to use a pre-trained model on similar tasks and a larger dataset, followed by transfer learning to transfer the knowledge from the first model. For this, we tested two different models:

- The guideline model: one represents a simple encoder-decoder structure. Here's below details all specific task for this architecture:
  - **Encoder**: Three convolutional blocks (use simple convolution layers (Conv2d)) extract features, with each block followed by max-pooling for downsampling.
  - **Decoder**: Two blocks with bilinear upsampling combine with skip connections from the encoder to reconstruct the segmentation mask. These two blocks are based on upsampling and adding encoder features using basic bilinear interpolation. Fewer layers in the decoder pipeline might limit its ability to capture fine details.
  - **Drawbacks**: No batch normalization or pre-trained backbone limits its ability to generalize, particularly for small datasets.
  - In the output, we directly apply a final convolution layer to the last decoder block, generating the segmentation output.
- MobileNetV2 is a convolutional neural network developed by Google, known for its lightweight and efficient architecture. We chose to use MobileNetV2 to perform transfer learning as a backbone (or encoder) in our segmentation task. To achieve this, we combine MobileNetV2 with our own model serving as a decoder. This approach follows the principle of the U-Net architecture, where the encoder layers are connected to the corresponding decoder layers through skip connections. This helps retain low-level information (fine details) while capturing high-level information (global features).

  The main reason for choosing MobileNetV2 is that it has been pre-trained on ImageNet, a vast database containing more than 14 million images across 1000 different categories, including several categories specific to dogs and cats. Thanks to this training, MobileNetV2 can efficiently extract complex visual features such as shapes, textures, and characteristic patterns of animals.

  By using MobileNetV2 as an encoder, we benefit from a model already competent in recognizing and extracting relevant information from images. This allows us to improve the performance of our segmentation task without the need for full training from scratch. This method not only saves time but also achieves higher accuracy, even with a reduced data volume.

Ultimately, we chose to keep MobileNetV2, which delivered significantly better performance.

2.3 Other experiments

To improve our model, we decided to preprocess the images using the wavelet transform. This technique decomposes an image into different frequencies, highlighting edges and fine details. The intuition behind this approach was to combine the preprocessed image with the original RGB image to provide the model with enriched input containing both color information and prominent edges. This was theoretically expected to help the model better identify the edges of animals and thus improve the quality of the segmentation masks.

However, this approach did not improve the predictions for several reasons:

- **Redundant information**: The deep layers of convolutional models like U-Net are already effective at automatically learning edges and complex image features. Adding an explicit edge map introduced redundancy, which did not provide any new useful information to the model.
- **Additional noise**: The wavelet transform can sometimes amplify noise in the images, which can hinder the model. Instead of clarifying object edges, the additional noise made it more difficult for the model to learn relevant features.
- **Poor information fusion**: We encountered some difficulties in merging the RGB images with the wavelet-transformed images. Although we eventually succeeded, if the fusion was not performed correctly, the model might fail to extract meaningful information.

2.4 Improvement Ideas

Although many improvements have been made, there are still many paths left to explore. Here are some research ideas we had but have not yet been able to try:

- Run our model in low resolution to get a less precise but more realistic result, then increase its resolution by passing it through another denoising model. This process would smooth the edges and produce a result very close to what we would get with a mask directly generated in high resolution.
- Adjust the contrast of the input images during training and testing by stretching the contrast to fully cover the entire spectrum. This would improve the visibility of the image and make it easier to see the edges when there is little difference between the object color and the background.

3 Conclusion

To conclude, by following the guidelines and conducting our own research, we successfully improved our model to achieve a satisfactory result. However, many improvements are still possible and could be explored further.